Analizador de Logs en Python: resumen del trabajo realizado

Estudiante: Jonathan Alberto | **Curso:** AI for Developers Jr. 4 (KODIGO Academy) | **Actividad:** IA como herramienta, no como autor | **IA de apoyo:** Claude

1. Objetivo y qué se construyó

Se desarrolló un script en Python que lee un archivo de texto con eventos de una aplicación (un log), analiza cada línea y muestra en consola un resumen: total de eventos, cantidad por nivel de severidad (INFO, WARNING, ERROR) y número de líneas mal formateadas o incompletas, con su número de línea y el motivo. El archivo se elige con un selector gráfico, indicando la ruta por argumento o escribiéndola. El script no usa librerías externas y maneja errores de archivo (inexistente, carpeta, sin permisos o vacío) con mensajes claros. La línea bien formada tiene la forma `[NIVEL] AAAA-MM-DD mensaje`.

El punto central de la actividad es el criterio humano: la IA puede proponer código, pero **las reglas de validación las define el estudiante**. Por eso el trabajo se organizó así: primero yo definí las reglas, después la IA escribió el código siguiendo esas reglas, y finalmente se probó y corrigió.

2. Reglas de validación y quién decidió cada una

#	Regla	Origen
1	Nivel válido pero **sin fecha**: cuenta como evento y también como mal formateada	Decisión mía
2	**Sin nivel válido** (sin corchetes o con un nivel como [DEBUG]): mal formateada y no cuenta como evento	Decisión mía
3	**Fecha incorrecta o imposible** (ej. 2025-13-45): toda la línea es mal formateada y no cuenta como evento	Decisión mía (la IA propuso la interpretación de no contarla como evento; la confirmé)
4-7	Sin mensaje: mal formateada; niveles sensibles a mayúsculas; líneas vacías ignoradas; fecha estricta AAAA-MM-DD	Propuestas de la IA, confirmadas por mí
8	Algo «parece fecha» solo si tiene año de 4 dígitos (corrige el caso de las versiones, ver sección 4)	Decisión mía

3. Reparto de responsabilidades entre la IA y yo

La IA escribió el código a partir de mis reglas y propuso: las expresiones regulares, la división en funciones (`parse_line`, `analyze`, `print_report`, entre otras), el uso de `datetime` para rechazar fechas imposibles como 2025-02-30, el selector de archivos, los logs de prueba y las pruebas automatizadas. **Yo** definí las reglas de validación, confirmé los valores por defecto que la IA propuso para los casos que no había definido, decidí qué casos borde manejar y cuáles dejar como limitación, elegí cómo corregir una sugerencia mejorable y revisé los resultados. En el código cada regla está comentada como «DECISIÓN DEL ESTUDIANTE».

4. Ejemplo concreto: una sugerencia mejorable y su corrección

Para distinguir entre «no hay fecha» y «fecha mal escrita», la IA propuso una expresión regular demasiado amplia: aceptaba cualquier forma número-separador-número-separador-número. La línea `[INFO] 1.2.3 released` (un número de versión, sin fecha) cumplía esa forma, así que se descartaba como «fecha inválida» y no contaba como evento, mientras que `[INFO] 404 pages not found`, que tampoco tiene fecha, sí se trataba bien. Dos líneas con el mismo defecto recibían tratamientos distintos.

La propia IA advirtió esta limitación y yo la comprobé ejecutando el programa. Entre tres alternativas elegí la más coherente con mi regla 3: exigir un año de 4 dígitos. Primero actualicé las pruebas con el comportamiento que quería y comprobé que fallaban (2 fallos); después cambié la expresión y pasaron las 16 pruebas.

```
ANTES (IA):   r"^\d{1,4}[-/.]\d{1,2}[-/.]\d{1,4}"
DESPUES (yo): r"^(?:\d{4}[-/.]\d{1,2}[-/.]\d{1,2}|\d{1,2}[-/.]\d{1,2}[-/.]\d{4})(?!\d)"
```

5. Evidencia de pruebas

Se prepararon cuatro archivos de prueba. Los resultados esperados se calcularon **a mano**, leyendo cada log y aplicando las reglas, antes de ejecutar el script; luego se compararon con lo obtenido. En todos los casos coincidieron.

Archivo	Descripción	Eventos (INFO/WARN/ERROR)	Mal formateadas	Resultado
log_bueno	10 líneas bien formadas	10 (5 / 2 / 3)	0	Esperado = obtenido
log_con_errores	13 líneas con errores: sin fecha, [DEBUG], texto suelto, fechas inválidas, sin mensaje, nivel en minúscula	5 (2 / 2 / 1)	8 (1 también es evento)	Esperado = obtenido
log_casos_borde	14 líneas límite: BOM, sangría, 2025-02-30, año bisiesto, tildes, versión 1.2.3, fecha con T	10 (6 / 1 / 3)	5 (2 también son evento)	Esperado = obtenido
log_vacio	Archivo sin contenido	-	-	Aviso claro, sin errores

Además, `test_analizador.py` incluye **16 pruebas automatizadas** que verifican los tres logs, cada regla, las líneas vacías y los errores de archivo. Todas pasan.

6. Limitaciones conocidas

No se manejan los eventos de varias líneas (como los stack traces); la hora no se valida (queda como parte del mensaje); el archivo se carga completo en memoria, por lo que no es ideal para logs muy grandes; y los niveles distintos de INFO, WARNING y ERROR se consideran inválidos. Estas decisiones están documentadas en el README.

7. Qué aprendí

Aprendí que la IA es muy rápida para producir código, pero ese código contiene supuestos escondidos: decidir qué es una línea válida es una decisión de negocio, no técnica, y solo yo podía tomarla. También aprendí que una sugerencia puede funcionar en los casos obvios y fallar en los límites, como pasó con `1.2.3`, por lo que conviene probar con datos incómodos y escribir el resultado esperado antes de ejecutar. Mi responsabilidad no es escribir cada línea, sino definir las reglas, revisar lo generado, probarlo y poder explicar por qué el código se comporta como lo hace.

8. Archivos entregados

`analizador_logs.py` (script), `test_analizador.py` (pruebas), `README.md` (reglas y evidencias), `REFLEXION_TECNICA.md`, la carpeta `logs_prueba/` con los cuatro logs, y este resumen.